PROJECT DOCUMENTATION

Automated Intern Offer Letter Generation & Mailing System

Full Stack Internship Project Brief

Project Type	Full Stack Web Application
Assigned To	Full Stack Intern
Document Version	1.0
Prepared For	Internal HR / Engineering Automation Initiative

1. Project Overview

The Automated Intern Offer Letter Generation & Mailing System is an internal HR-tech tool that automates the process of generating and sending offer letters to candidates who have cleared their interview rounds. Instead of an HR executive manually drafting each offer letter in Word/PDF and emailing it individually, this system will let HR fill in a candidate's details through a web form, auto-generate a professional offer letter (PDF) from a template, and automatically email it to the selected candidate — all in a single click.

2. Vision

To build a scalable, reusable internal automation platform that eliminates manual, repetitive HR documentation work — starting with offer letters — and can later be extended to other HR workflows (relieving letters, appraisal letters, certificates, etc.), reducing turnaround time from hours to seconds and eliminating human error in candidate communication.

3. Purpose / Problem Statement

Current manual process:

1. HR manually opens a Word/Google Docs offer letter template.
2. HR manually edits candidate name, role, date of joining, stipend/salary, etc.
3. HR exports it to PDF.
4. HR manually composes an email, attaches the PDF, and sends it.
5. This is repeated for every single candidate — time-consuming and error-prone (wrong name, wrong CTC, wrong date carried over from the last letter, etc.).

Problems this causes:

- Delayed offer rollout to candidates (bad candidate experience, risk of losing candidates to other offers).
- Inconsistent formatting/branding across letters.
- No central record of who was sent what, and when.
- Manual errors in critical legal/financial fields (CTC, designation, joining date).

What this project solves:

- One-click, templated, error-free offer letter generation.
- Automatic email dispatch the moment the letter is generated.
- A central dashboard/record of all offers issued (audit trail).
- Foundation for future HR process automation.

4. Goals & Objectives

- Build a form-based interface for HR to input candidate & offer details.
- Auto-generate a branded, professional PDF offer letter from a template using dynamic data.
- Auto-send the generated offer letter via email to the candidate immediately after generation.
- Store a record of every offer letter generated (candidate info, status, timestamp, letter copy).
- Provide HR with a dashboard to view/search/download past offer letters.
- Ensure the system is secure (only authorized HR/Admin users can generate letters).

5. Scope

In Scope (Phase 1 – Intern Deliverable)

- HR/Admin login (authentication).
- Candidate offer detail entry form.
- Dynamic offer letter PDF generation from a template.
- Automated email dispatch with PDF attached.
- Database record of generated offers.
- Admin dashboard to view offer history and resend/download letters.

Out of Scope (Future Phases)

- E-signature integration (candidate digitally accepting offer).
- Multi-level approval workflow (Manager → HR Head sign-off before sending).
- SMS/WhatsApp notification integration.
- Integration with HRMS/ATS (e.g., Darwinbox, Keka, Greenhouse).
- Multi-template support (different letters for different departments/roles).

6. User Roles

Role	Permissions
Admin (HR)	Login, create/generate offer letters, view all records, resend emails, download PDFs
(Optional future role) Approver	Reviews and approves offer before it is sent
Candidate	Only a recipient — receives the email with the offer letter; no login required in Phase 1

7. Technology Stack (Suggested)

This is a good stack for a MERN-based full stack intern project. Feel free to substitute per your team's existing stack.

Frontend

- React.js (with Vite) – UI for HR to fill candidate details & view dashboard
- Tailwind CSS – styling
- Axios – API calls
- React Hook Form + Yup/Zod – form handling & validation
- React Router – navigation

Backend

- Node.js + Express.js – REST API server
- JWT (jsonwebtoken) – authentication for HR/Admin login
- bcrypt – password hashing

Database

- MongoDB (with Mongoose) – store candidate, offer, and email-log records
- Alternative: MySQL/PostgreSQL with Prisma/Sequelize, if your team prefers SQL

PDF Generation

- Puppeteer (renders an HTML/CSS offer letter template to PDF) — recommended, gives full design control
- Alternative: pdf-lib or pdfkit if a simpler, code-based PDF is preferred

Email Automation

- Nodemailer – sending emails via SMTP
- Gmail SMTP / SendGrid / Mailgun / Amazon SES – email delivery provider (SendGrid or SES recommended for production reliability & deliverability tracking)

Others

- dotenv – environment variable management
- Multer (optional) – if letterhead/logo needs to be uploaded
- Cloudinary / AWS S3 (optional) – to store generated PDF copies
- Postman – API testing
- Git & GitHub – version control
- Render or Vercel – deployment

8. System Architecture (High-Level Flow)

```
HR Login
   |
   v
Fill Candidate & Offer Details Form
(Name, Email, Role, CTC, DOJ, Manager, etc.)
   |
   v
Submit -> Backend API (/api/offers/generate)
   |
   v
Backend fetches Offer Letter HTML Template
-> Injects dynamic candidate data
-> Converts HTML to PDF (Puppeteer)
   |
   v
Save Offer Record in Database
(candidate info + PDF path/URL + status: "Generated")
   |
   v
Trigger Email Service (Nodemailer)
-> Attach generated PDF
-> Send to candidate's email
   |
   v
Update Database status -> "Sent"
   |
   v
HR Dashboard shows updated status + downloadable copy
```

9. Core Modules & Features

Module 1: Authentication

- HR/Admin registration (or seeded admin account) & login
- JWT-based session handling
- Protected routes (only logged-in HR can access the dashboard/generate letters)

Module 2: Offer Letter Generation

Form fields (minimum):

- Candidate Name
- Candidate Email
- Position/Designation
- Department
- Date of Joining
- Internship Duration / Stipend (or CTC, if full-time)
- Reporting Manager
- Offer Issue Date

"Generate Offer Letter" button → generates PDF preview before sending (nice UX addition)

Module 3: PDF Template Engine

- One master HTML/CSS offer letter template with placeholders (e.g., {{candidateName}}, {{designation}}, {{joiningDate}}) styled with company branding/letterhead
- Backend replaces placeholders with real data and renders to PDF

Module 4: Automated Email Dispatch

On successful generation, automatically:

- Compose email (subject: "Your Internship Offer Letter – [Company Name]")
- Attach the generated PDF
- Send to candidate's email
- Log delivery status (Sent / Failed) with error handling & retry option

Module 5: Offer Records Dashboard

- Table view of all offers generated: Name, Role, Date, Status (Sent/Failed/Pending)
- Search & filter by name/date/status
- Download or resend option for each record

10. Suggested Database Schema (MongoDB Example)

User (HR/Admin) Collection

```
{
  name: String,
  email: String,
  password: String (hashed),
  role: "admin",
  createdAt: Date
}
```

Offer Collection

```
{
  candidateName: String,
  candidateEmail: String,
  designation: String,
  department: String,
  dateOfJoining: Date,
  stipendOrCTC: String,
  reportingManager: String,
  offerIssueDate: Date,
  pdfUrl: String,
  emailStatus: "Pending" | "Sent" | "Failed",
  generatedBy: ObjectId (ref: User),
  createdAt: Date
}
```

11. Non-Functional Requirements

- Security: Passwords hashed with bcrypt; JWT-protected routes; input validation & sanitization to prevent injection attacks.
- Reliability: Email failures should be logged and retryable, not silently dropped.
- Performance: PDF generation & email dispatch should be handled asynchronously (don't block the API response — consider a queue for scale, e.g., Bull/BullMQ with Redis, as a stretch goal).
- Usability: Simple, clean HR-facing UI; no technical knowledge required to use it.
- Maintainability: Code should be modular (controllers/services/routes separated), well-commented, and follow consistent naming conventions.

12. Suggested Project Folder Structure

```
offer-letter-automation/
|-- client/                     # React frontend
|   |-- src/
|   |   |-- components/
|   |   |-- pages/
|   |   |-- services/ (axios API calls)
|   |   `-- App.jsx
|-- server/                     # Node/Express backend
|   |-- config/ (db.js, mailer.js)
|   |-- controllers/
|   |-- models/
|   |-- routes/
|   |-- templates/ (offerLetter.html)
|   |-- utils/ (pdfGenerator.js, emailSender.js)
|   |-- middleware/ (auth.js)
|   `-- server.js
|-- .env.example
`-- README.md
```

End of Document